2027年热门会议推荐

具体咨询:白老师 13516196409

智驱产品技术领航

AI 时代产品和软件开发前沿系列峰会

核心定位

锚定AI 技术革新前沿,聚焦大模型与智能体工程化落地,解构 AI 时代软件工程架构革新路径,打通 AI 产品从技术突破到产品商业化验证的全链路。

关键词

- 大模型驱动
- 技术趋势
- Agent工程化
- 架构革新
- AI产品0-1
- 商业化验证

4月 PM×AI 产品力领航者大会

2027年4月23-24·杭州

详情查看官网
演讲申报 | 参会购票

6月 Agentic AICon 智能体应用与架构工程大会

2027年6月18-19·上海

详情查看官网
演讲申报 | 参会购票

11月 Agentic AICon 智能体应用与架构工程大会

2027年11月5-6日·深圳

AI 赋能提质增效

QECon全球软件质量&效能大会

核心定位

聚焦 AI 技术在软件研发全流程的落地应用,以“质量保障&效能提升”为双驱动,助力研发团队实现软件研发的智能化转型。

关键词

- AI 赋能研发
- 效能提升
- 测试驱动
- 平台工具链
- 质量保障
- .....

5月 QECon 全球软件质量&效能大会

2027年5月21-22·深圳

9月 QECon 全球软件质量&效能大会

2027年9月10-11日·上海

10月 QECon 全球软件质量&效能大会

2027年10月15-16日·北京

演讲申报 | 参会购票
详情查看官网

生产级Agent设计蓝皮书·行业案例卷

"蓝皮书共建计划"启动——行业与场景专家 招募中

白皮书定义模式，蓝皮书记录落地。
把你的生产案例，写进行业参照系。
你的案例，很可能是这个行业第一份模式应用实践指南。

交流地点：《艾普·AI生产力研究院》展位咨询

扫码参与



共建计划

以真实案例为基础，持续组织行业与场景的 Agent 设计模式应用撰写。蓝皮书不写概念，只记录具名系统——它面对什么约束、踩过什么失败、为什么这样选架构、哪些经验可以迁移到你的行业。

我们招募这样的行业与场景专家

主导或深度参与过 Agent 系统建设，有真实跑在生产环境的案例；
在强监管、高可靠或大规模部署约束下做过工程取舍（金融、制造、政务、医疗、能源、电商等皆宜）；
愿意把一段经验，沉淀成可评审、可迁移的模式案例。

参与共建，你将获得

署名蓝皮书案例贡献者，成果进入 ADPS 公共技术资料长期存档；
用白皮书的模式语言系统梳理自己的实践，让团队经验成为行业参照；
参与共同体模块研讨与模式评审，共同界定下一版模式的边界；
与已在共同体的腾讯、蚂蚁、百度、京东等一线专家跨行业互审。

《企业级 Agent 设计白皮书》发布

纸质版 & 电子版 同步上线

领取地点:《艾普·AI生产力研究院》展位

Agent 正在从「会回答问题的模型」变成「能参与业务流程的系统」。模型仍是引擎,但决定它在生产中稳不稳的,是周围这套工程结构。本白皮书给出智能体工程的可复用架构词汇与评审标准。



27	7×6	3	5	6
矩阵模式规范	认知功能 × 执行拓扑	横切工程面 X1–X3	工程契约	步选型法

智能体工程的「GoF设计模式」

27个可复用Agent设计模式双轴框架

企业级智能体架构评审与选型实战手册

双轴框架

纵轴 · 认知功能

Agent 在做什么类型的工作

感知 Perception | 记忆 Memory | 推理 Reasoning | 行动 Action | 反思 Reflection | 协作 Collaboration | 治理 Governance

横轴 · 执行拓扑

数据与控制怎么流动

链式 · 顺序 | 路由 · 分流 | 并行 · 并发 | 编排 · 协调 | 循环 · 迭代 | 层级 · 分层

获取白皮书电子版

艾普·AI生产力研究院

Artificial Intelligence Institute of Productivity

Artificial Intelligence Institute of Productivity · 中立 · 开放 · 共创

面向 AI 原生软件工程的开放研究与实践组织

驾驭智能·定义 AI 时代新质生产力

深度承接《"人工智能+软件"专项行动实施方案》的专业化研究与交流平台

软件质量与效能 | **智能体工程与应用** | **六大研究方向** | **五大核心模块**

艾普 · AI 生产力研究院（AIIP）是一个中立、开放、共创的专业化研究与交流平台。研究院依托在软件质量与效能、智能体工程与应用等前沿领域的长期实践积累与前瞻研究底蕴，汇聚资深技术专家与企业研发团队等各方力量，共同推进行业能力体系建设、参考标准研究与评估框架研制，致力于成为支撑智能软件产业新质生产力发展的专业化交流平台。

更多详情请查看官网：https://aiip.org.cn/

关注官网

申请加入

与先行者同行，共赴 AI 生产力浪潮

六大研究方向

AI for SE

智能软件工程

AI赋能软件开发全流程、AI Coding 生产力与质量保障、智能测试工程，对标智能编程工具链与智能化技改。

度量体系

质效度量与治理

AI 质效度量模型、Token 成本治理、ROI 分析框架，以及成熟度评估参考框架的研究与共建。

ADPS

Agent 设计模式

Agentic 设计模式、多智能体架构、AI 系统技术栈与最佳实践沉淀，对标技能包与应用商店。

FDE

前线部署工程师

工程实践方法论、行业场景适配、企业落地案例与效能提升路径，对标就业友好与重点行业落地。

SE for AI

AI 工程化落地

AI 系统全生命周期工程化方法，可靠性与质量保障，以及 MLOps / LLMOps 交付与治理框架。

AI for X

AI 生产力

AI深度赋能千行百业，从软件研发走向办公、运营、数据、科研等真实业务价值场景，把工程能力转化为可度量的生产力。

AI 质效宣言

发起单位：QECon组委会 QECon Organizing Committee

从共识到行动，让宣言融入每一次研发实践

回归研发本质 **重塑人机协作** **推动行业共识**

协作共生 高于 局部取代
长效质量 高于 短期提效
适应进化 高于 固化流程

价值创造 高于 技术崇拜
数据驱动 高于 经验依赖
开放共享 高于 封闭自建

加入共建计划
获取电子画面

两层能力递进 · 一层工程底座

AI 增强软件研发全景架构

AI 嵌入 SDLC · 效能提升

• 人主导 • AI 协同

需求	设计	开发	测试	CI / CD	运维 / 运营
行业调研与趋势分析	代码理解与逻辑探测	代码生成与缺陷解决	测试用例与数据生成	构建与部署脚本生成	告警聚合与摘要生成
创意启发与功能探索	技术选型与架构设计	氛围编程与规约编程	测试脚本修复与自愈	流水线失败诊断与修复	故障分析与定位
需求分析与规格细化	接口设计与数据模型生成	单元测试生成与执行	端到端测试生成与维护	无人值守部署与发版	故障处理与系统自愈
需求检查与内容评审	技术方案评审与风险分析	代码评审与问题修复	缺陷发现与根因分析	依赖升级与漏洞修复	用户体验与反馈洞察

AI 研发范式变革 • 效能跃迁

• AI 主导 • 人监督

工程实践
- 驾驭工程（模型决定上限，Harness 提升底线）
- 知识工程（跨仓库代码与文档索引 · 沉淀复用）
- 验证闭环（生成 → 验证 → 修正）
- 质量门禁（构建多层次质量防护网）

流程重构
- 意图驱动、面向结果的交付（替代多阶段接力）
- 对齐在前、执行在后（人谈清边界，AI 自主实现）
- 需求即规格、证据即信任（规格成为人机契约）
- 快速原型、持续迭代（沿用敏捷 · 精益研发思想）

组织重塑
- 交付层抹平前后端分工（跨栈交付 · 功能负责人）
- 架构师专注标准与门禁（定标准 · 守门禁）
- 设计师与产品自主交付（部分需求不经研发）
- 先锋队试点后规模复制（先跑通再逐步推广）

效能度量
- 结果面：多 · 快 · 好 · 省
- 过程面：渗透率 · 成熟度 · 使用效果
- 数据源：代码 · 会话 · 配置 · 需求 · 监控
- 北极星：需求交付周期 · 单位 token 有效价值

AI 工程底座 • SE4AI（Software Engineering for AI）

• 自下而上 • 决定能力上限

模型层
- 多模型接入与路由（模型网关 · 按能力与成本择优）
- 能力分级与降级（强弱模型分级 · 故障回退）
- 领域微调与私有化（微调训练 · 内网部署）
- 模型评测与选型（基准跑分 · 场景实测对比）

Agent 层
- Agent 架构与编排（单/多 Agent · 规划与控制流）
- 上下文工程与记忆（上下文组装压缩 · 长短记忆）
- 运行时沙盒与工具接入（隔离执行 · MCP 与技能调用）
- 灰度发布与回滚（能力评估达标才放量 · 版本化 · 自动回退）

生态层
- 技能与智能体市场（技能复用 · Agent 共享流通）
- MCP Hub 与工具生态（工具目录 · 权限与调用治理）
- 知识资产与语料沉淀（业务知识 · 代码知识 · 过程知识）
- 检索服务与知识图谱（语义检索 · 图谱关系推理）

贯穿三层 | **安全与治理** 数据脱敏 · 提示注入防护 · 最小授权 · 输出合规 | **可观测性** 决策链路追踪 · 上下文溯源 · 失败归因 | **能力评估** 任务成功率 · 人工接管率 · 单任务成本

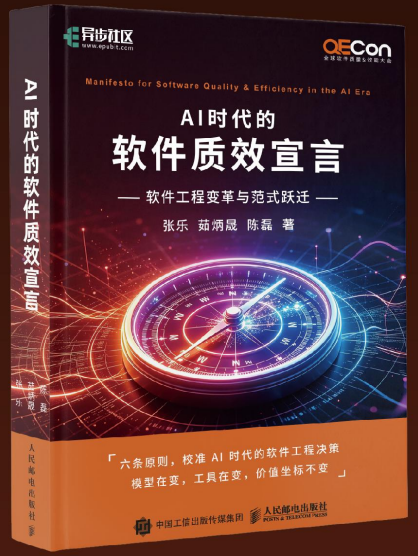
为每一位实践者提供一组稳定的价值坐标——当工具在变、模型在变、最佳实践在变的时候，这些原则帮你判断什么是不变的、什么是值得坚守的。

AI 时代的软件质效宣言

新书发布

— 软件工程变革与范式跃进 —

扫码订阅



系列电子书结构·三篇十九章 (道 · 术 · 势 —— 一条完整的 AI 研发变革路径)

道

第一篇 · 解读篇 (1-8 章)

确立价值观与原则：理解 AI 时代软件质效的本质，重建研发协作的信念底座，是全书的「道」。

术

即将发布

第二篇 · 单点提效篇 (9-14 章)

覆盖需求分析 → 架构设计 → 编码实现 → 测试验证 → CI/CD → 运维运营的关键环节，打通 AI 赋能研发全链路，是全书的「术」。

势

即将发布

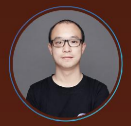
第三篇 · 范式篇 (15-19 章)

当 AI 贯穿整条研发链路之后，组织形态本身将如何变革？从团队结构到协作范式，预见未来，是全书的「势」。

—— 主编 ——



张乐
腾讯
智能化软件工程
资深技术专家

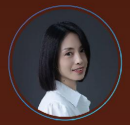


茹炳晟
腾讯研究院
高级技术顾问



陈磊
前京东测试架构师

—— 作者 ——



王玉霞
北京兴云数科
过程改进专家



李佳奇
去哪儿旅行
技术总监



牛万鹏
百度
Comate研发经理



张晔
腾讯端服务质效
技术负责人



张涛
快手质量通道委员



付求爱
华为云
AIOps技术负责人



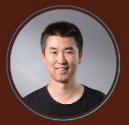
李萌
珊瑚智成
创始人&CEO



张刚
上海英慕科技
资深技术专家



张兆鑫
OPPO AI架构师



王磊
华为 AI技术专家

主办方：智盟创课

智盟创课（北京）科技有限公司（简称“智盟 Stu.”），作为专业IT产研培训服务供应商与AI时代数字化人才发展伙伴，为科技研发型企业及各类组织提供系统性、分层次、重实战的人才培养解决方案，同时搭建行业前沿技术交流平台，致力于成为企业数字化转型进程中人才培养与业务创新的核心支撑力量，推动组织实现成功转型及经营业绩持续增长。

同时，智盟发起并组织三大 IT 领域品牌会议：QECon 软件质量效能大会、SECon+AgentX 智能时代软件工程技术大会、PMxAI 产品力领航者大会、Agentic AICon 智能体应用与架构工程大会，助力产研中心深化数字化转型、聚焦业务价值达成，在产品研发的快速交付与高质量发展之间找到平衡，打通企业全维度学习视角，最终实现人才能力提升与企业竞争优势增强的双重目标。



详情咨询: 010-65798049

多元化学习形式



核心培训方向



软件研发赋能

大模型前沿	架构设计	数据库	产品经理	持续运维	质量管理
移动开发	组织管理	云计算	开发编程	敏捷开发	大数据

Agentic AICon

基于 OpenTelemetry eBPF Instrumentation 的 Sandbox 可观测性

张海彬（古琦） | OBI Approver · GitHub @NameHaibinZhang

让每一次沙箱执行透明可见 —— 零代码、零 SDK、零重启



张海彬（古琦）

OBI 核心贡献者 · GitHub @NameHaibinZhang

OBI（opentelemetry-ebpf-instrumentation）Approver,
OpenTelemetry Go Compile Instrumentation Maintainer,

OpenTelemetry Maintainer,

阿里云可观测研发工程师。

目录

01 黑盒之困：Sandbox 为何难观测

02 破局思路：OBI 站在内核与应用之间

03 沙箱落地：Sidecar 接入与 GenAI 语义

04 实战闭环：三个真实问题与收益

05 总结与展望：可观测是下半场

PART ONE

黑盒之困：Sandbox 难在哪

01

Agent 把代码执行放进沙箱：隔离做得越好，沙箱内部就越像黑盒。

Sandbox 是什么：隔离边界与运行形态

Sandbox = 隔离边界 + 独立文件系统与网络 + 可编程生命周期 —— 把不可信执行关进一个可随时丢弃的盒子。

隔离层级：强度 / 启动开销 / 兼容性的三角

隔离形态	隔离边界	启动	定位
进程级 chroot	共享内核 · 仅命名空间	<10ms	开发调试
容器 runc	namespace + cgroup	~100ms	普通微服务
用户态内核 gVisor	自实现 syscall 层	~200ms	中等隔离
MicroVM	独立 guest 内核	百 ms 级	AI Agent
WASM / Pyodide	运行时指令级	<10ms	轻量代码

K8s Sandbox：三类接口，三层可观测

控制面 · 创建 / 销毁
CR / E2B SDK 声明镜像、资源、超时
→ 拉起时延 · 失败率 · 并发水位

执行面 · exec / 文件
命令、代码执行与文件读写
→ 命令耗时 · 退出码 · I/O 字节

状态面 · 暂停 / 恢复
内存快照、Checkpoint / Restore
→ 唤醒时延 · 状态一致性

隔离层级决定「能不能采」，接口形态决定「采什么」 —— 两者叠加，就是下一节要拆的黑盒。

Agent 时代，沙箱成为事实标配

大模型现场生成的代码不可信、不可预测，绝不能与生产业务同容器运行 —— 沙箱是 Agent 基础设施的标配底座。

MicroVM 级隔离

每个沙箱独享内核
计算 / 网络 / 存储端到端隔离
异常行为被锁在「箱子」里

大规模低延迟弹性

最高每分钟 15000 个沙箱
Warm Pool 百毫秒级拉起
镜像缓存提速 90% 以上

状态保持

休眠内存完整保留，1~10s 唤醒
Checkpoint / Restore 可存可迁可克隆
按秒计费，随用随停

K8s Sandbox：Agent 执行的标准底座

兼容 E2B SDK 与 K8s Sandbox CR；覆盖 Code Interpreter、Browser Use 等场景。公开案例：MiniMax 10 秒内拉起 5000 个沙箱。

隔离解决了「敢不敢跑」，但「跑得怎么样」仍是迷雾 —— 这正是可观测要回答的问题。

越安全，越像黑盒：三条路都走不通

1. SDK 埋点埋不进去 —— 沙箱代码由模型即时生成、五花八门不可预置；Agent 框架横跨 Python / Node.js 多语言，无法要求模型「生成代码时顺便埋点」。

2. 主机级采集无处安放 —— 托管 Sandbox 服务无常驻用户节点，DaemonSet 式主机监控无立足之地；MicroVM 强隔离堵死「从宿主机看进去」。

3. 生命周期太短 —— 百毫秒创建、用完即毁、随时休眠；等你登录排查，现场早已销毁。以主机为中心的监控跟不上「朝生暮死」的节奏。

黑盒的三重代价

Agentic AICon

性能

任务失败了，
慢在模型推理、
工具超时，
还是代码报错？
无从定位。

成本

token 账单上涨，
消耗在哪个环节？
哪类调用最贵？
传统 APM
不暴露 token 字段。

安全

沙箱内不可信代码
访问了哪些外部地址？
只能翻网络日志
人肉比对，
缺乏审计底账。

结果：性能、成本、安全三笔账，全都算不清。

进程级监控：沙箱里跑了什么

把沙箱当 API 记账，能算钱、能算成功率；但要回答「到底干了什么」，还差一层进程视图。

AWS AgentCore 已提供（会话级）

会话 / 资源 / 日志

会话指标 · Invocations / Throttles
错误 · SystemErrors / UserErrors
延迟 · Latency / Duration
资源 · vCPU-Hours / GB-Hours
（分钟级，可能延迟至 60 分钟）
Span · Start / Invoke / Stop / Expire
日志 · 代码与响应的 payload

仍然缺失（进程级）

执行行为

exec / fork / exit · 进程生命周期
命令行与参数 · 外联审计依据
退出码 · 失败归因到具体命令
stdout / stderr · 输出与报错
文件 I/O · 谁写了 /tmp、谁读凭证
进程级 CPU / RSS · 明细而非整箱

缺 结果：算得清「花了多少」，算不清「干了什么」 —— 提案 #3210 正是补这一层。

AWS 的应用日志能看到「提交了什么代码」，但看不到「代码跑出了哪些进程」。

Code Runner 与 Browser：两套证据

同一个沙箱，两类负载的黄金信号完全不同 —— 观测不能一套打天下。

Code Runner · 短生命周期

已有：会话指标 + vCPU / GB-Hours
已有：Invoke 的代码与响应 payload
缺口：进程树 · 退出码 · stdio
黄金信号：执行时长 · 超时率
一次执行几个进程，谁失败了?

Browser · 长生命周期

已有：Live View（DCV）实时接管
已有：会话录制 → DOM / console / CDP / 网络
已有：TakeOverCount · 会话时延
缺口：页面事件与 Agent trace 关联
黄金信号：导航失败 · console error

→ Runner 看退出码，Browser 看导航与 console —— 最终都要挂回同一条 trace，才能告警而不只是回放。

AWS 的浏览器录制很全，但它是离线录像：能回看，不能聚合、不能告警 —— 在线指标要靠旁路持续采集。

PART TWO

破局：站在内核与应用之间

02

OBI 不进入应用，而在内核与库函数层拦截通信、解析协议语义，直接输出标准 OTel 遥测。

OBI：内核级透镜，零代码可观测

OpenTelemetry eBPF Instrumentation（OBI），OTel 原生、源自 Grafana Beyla：在内核拦截并解析应用通信，零代码、零 SDK、零重启。

应用可观测
Trace + RED 指标
覆盖 Web / DB / MQ / GenAI

网络可观测
L3/L4 流、TCP RTT
服务拓扑与健康度

日志增强
自动注入 trace_id
Trace 与 Log 关联

24	20+	5	0
协议插桩	GenAI Provider & MCP	语言深度插桩	行代码改动

标准 OTLP 导出 → 任意 OpenTelemetry 兼容后端；K8s DaemonSet / Sidecar 自动服务发现。

数据通路：内核字节 → OTel Span



加密流量也能看清

- 在加密前 / 解密后捕获明文（uprobe libssl）—— 无需密钥、无 MITM、无 LD_PRELOAD。
- 大缓冲拼接：从 ring-buffer 分段还原 10KB+ 的完整 LLM JSON 响应体。
- 语言无关：任何走 TCP + OpenSSL/BoringSSL 的应用，插桩方式完全一致。

协议覆盖全景：三级瀑布式识别

Agentic AICon

Web & RPC
HTTP/1.x · HTTP/2 · gRPC
GraphQL
TLS uprobe 明文捕获
HPACK 头部解码

数据库 & 消息队列
MySQL · PostgreSQL
Redis · MongoDB
Kafka · MQTT · NATS
SQL 语句级解析

GenAI & GPU
OpenAI / Anthropic / Gemini
Qwen / Ollama / vLLM
MCP 协议原生解析
CUDA 内核发射跟踪

三级瀑布匹配：连接首包识别一次，结果全程缓存复用

① 内核注解
socket 级协议标记
一条连接只付一次成本

→

② 确定性解析
固定协议头字段精确判定
HTTP/2 · gRPC · SQL

→

③ 启发式推断
HTTP 路径与响应体形状
兜底未知 Provider

24 类协议 · 一套内核透镜：应用与依赖用什么，OBI 就能看什么。

跨语言上下文传播：Trace 不断链

Go 协程

uprobe 挂钩 runtime.newproc1
与 casgstatus 调度事件
沿最多 6 层父 G 回溯，
继承完整 trace 上下文
net/http · gRPC 全覆盖

Python asyncio

挂钩 Task.__init__ / task_step
等 4 个关键探针
跨 await / gather 传播 context
原生适配 uvicorn / FastAPI

跨进程 tpinjector

HTTP/1：sk_msg 重定向注入
HTTP/2：HPACK 头部改写
TCP Option kind=25 透传
跨进程边界不丢链路

沙箱代码由模型现场生成、语言与框架不可预知 —— **语言无关的上下文传播，保证「生成什么代码，链路都完整」。**

无需 SDK 注入 trace 头 —— 上下文在内核里被接续。

旁路的代价：eBPF 开销从哪来、怎么压

开销从哪里来

kprobe 高频触发 —— 每个收发包
两次内核函数挂钩，软中断可见增量
eBPF map 读写 —— 连接跟踪表查找
与协议注解的存取
用户态解析 —— Go 协程完成协议
解析与属性装配

OBI 怎么把开销压住

ring-buffer 替代 perf buffer，
内核 → 用户态近零拷贝
内核态只捕获，过滤解析全下沉用户态
discovery 策略按需挂载，只插桩目标进程
span 批量导出 + 采样，产生量可控

✓ 典型生产环境 CPU 开销为个位数百分比、随流量线性 —— 用个位数开销换全量旁路可见性，沙箱场景值得。

为什么 OBI 是沙箱场景的「唯一可行解」

Agentic AICon

维度	传统 SDK / APM	OBI (eBPF)
代码改动	手动集成 SDK	零改动 · 内核级
语言支持	每语言一个 SDK	任何走 TCP 的语言
部署方式	重建镜像 + 重启	Sidecar / DaemonSet 免重启
沙箱 / VM 边界	通常是黑盒	runc 节点级；microVM 需 guest 侧

对「代码由模型现场生成」的沙箱场景，零改造几乎是唯一可行的可观测解法。

Trace 之外：网络、日志、GPU 三路增强

网络 • L3/L4

TC hooks 记录每条网络流
反向 DNS + GeoIP 装饰对端
TCP RTT / 重传 → 依赖健康度

日志 • trace_id 注入

kprobe tty_write / pipe_write
向 stdout / stderr 注入 trace_id
日志与链路一键互跳

GPU • CUDA 视图

uprobe cudaLaunchKernel
捕获 grid / block 维度与耗时
为 GPU 推理补上执行视图

5 一份 OBI 部署 = Trace + RED 指标 + 网络流 + 日志关联 + GPU 视图 —— 五类数据同源导出。

全部旁路产出，应用零配合。

GPU 观测的三层视角：请求到硅片

Agentic AICon

只看 kernel 发射，会漏掉排队与利用率；只看 DCGM，又说不清是哪个请求慢 —— 三层数据拼成一条因果链。

推理请求层 • OBI

HTTP / gRPC 语义解析
TTFT · TPOT · tokens/s
gen_ai.request.model 归属
回答：哪个请求慢

CUDA 运行时层 • OBI

uprobe cudaLaunchKernel
kernel 名 · grid / block
单次发射耗时与间隔
回答：慢在哪段计算

GPU 硬件层 • DCGM

SM 利用率 · HBM 带宽
显存占用 · 功耗 / 温度
按 GPU UUID 聚合
回答：是不是卡在算力

关联键：Pod / container → GPU UUID → trace_id —— 三段证据指向同一现场，才是完整因果链。

OBI 不读 NVML：容器内直接 uprobe CUDA 用户态库，无需 GPU 设备权限；硬件指标由 DCGM exporter 补齐。

GPU 瓶颈定位：四类慢，四种证据

现象	关键指标	判定	处置
TTFT 高 • SM 低	排队时延 · GPU 利用率 < 30%	**请求排队 / 副本不足**	扩容 · 调 batch 窗口
TTFT 高 • SM 满	prefill 耗时 · 显存 / KV 占用	**算力或显存瓶颈**	量化 · 限 max_num_seqs
TPOT 慢 • kernel 密集	HBM 带宽 · kernel 发射间隔	**decode memory-bound**	前缀缓存 · 增大 batch
抖动大 • 利用率尖刺	kernel 发射耗时 · 同步次数	**小 kernel / 同步等待**	CUDA Graph · 算子融合

OBI 提供第 1、4 行的请求与 kernel 证据；第 2、3 行需要 DCGM / 推理框架指标 —— 按 GPU UUID 关联同一现场。

PART THREE

落地：Sidecar 即接入

03

在 K8s Sandbox 中以 OBI Sidecar 与沙箱相伴运行，
零代码、零重启完成接入，数据以标准 OTLP 导出。

端到端架构：从沙箱 Pod 到控制台

沙箱 Pod
Sandbox 容器
OBI Sidecar 相伴

→

k8s-cache 服务
gRPC 元数据缓存
Pod / NS 装饰

→

OTLP 导出
Traces / Metrics
标准开放协议

→

可观测后端
Jaeger / Prometheus
任意 OTel 兼容平台

Sidecar 与沙箱同生共死
沙箱销毁，采集器自动回收
零残留、零独立运维

k8s-cache 独立 gRPC 进程
缓存 K8s 元数据：降低 API server
压力、内存占用与冷启动时间

内核 eBPF 捕获 → 用户态 Go 解析 → 标准 OTLP —— 全链路无应用侵入。

部署形态：OBI Sidecar 与沙箱相伴

① 部署 OBI
Sidecar 随 Pod 注入

② 配置发现
discovery 规则锁定目标进程

③ 自动插桩
eBPF 挂钩 · 协议解析

④ 数据落地
OTLP → 任意 OTel 后端

```
# Sandbox Pod 追加 OBI Sidecar 容器
- name: obi
  image: otel/ebpf-instrumentation:latest
  env:
  - name: OTEL_EXPORTER_OTLP_ENDPOINT
    value: http://otel-collector:4318
  securityContext: { privileged: true }
```

eBPF 挂钩需内核权限（privileged 或 CAP_BPF / CAP_PERFMON / CAP_SYS_ADMIN）；全程不动应用代码。

运行时边界：OBI 到底装在哪

Agentic AICon

运行时	OBI 部署位置	为什么
runc 普通容器	**节点级 DaemonSet**	与宿主共享内核，节点侧覆盖全部容器
Kata / Firecracker	**guest 侧（沙箱内）**	microVM 独立内核，宿主看不到 guest 系统调用
gVisor	**暂不支持**	用户态内核，syscall 不走宿主内核路径
K8s Sandbox	**guest 侧 Sidecar**	托管服务无常驻用户节点，Sidecar 随 Pod 自动注入

「microVM 须 guest 侧」是唯一部署约束 —— 也正是 Sidecar 形态的用武之地。

能力清单：链路、指标与 GenAI 语义

第一层 · 经典应用监控

应用拓扑 / 调用链路
异常事务 / 慢事务 / SQL 分析
每次对外调用生成 Trace，
一条链看穿慢在推理还是工具

第二层 · AI Agent 可观测

LLM：模型 / token / 耗时 / 错误
Tool Call：自动解析工具调用
MCP / Embedding / Rerank /
向量检索：RAG 全链路覆盖

GenAI 语义规范属性（自动归一化）

gen_ai.request.model · gen_ai.usage.input_tokens / output_tokens · gen_ai.tool.name · mcp.method.name / session.id
自建推理（vLLM / LiteLLM）或 LLM 网关：配置中声明 Host，即按 OpenAI 兼容协议解析纳入视图。

GenAI 语义是怎么被识别出来的

Agentic AICon

URL 路径
/chat/completions

响应头
server · request-id

响应体形状
choices[] · usage

Provider 判定
openai / qwen / …

属性归一化
gen_ai.* 标准语义

Model & Operation

gen_ai.request.model
gen_ai.operation.name
system 判定：openai / anthropic / qwen / …

Token Usage & Cost

gen_ai.usage.input_tokens
gen_ai.usage.output_tokens
跨 Provider 口径归一化，
按应用 / 模型可聚合

Tools & MCP

gen_ai.tool.name + 参数摘要
mcp.method.name / session.id
tool call / embed / rerank
11 类子操作统一识别

11 类 GenAI 子操作走同一条识别链 —— 新增 Provider 只需扩展识别规则。

一份数据，回答三个问题

Agentic AICon

性能

任务慢在哪个环节？
模型推理 / 工具执行 /
外部依赖，
一条 Trace 定位。

成本

token 消耗在哪类调用？
按应用 / 模型 / 操作
拆分 token 与耗时，
成本可归因。

安全

不可信代码访问了谁？
全部外联旁路记录，
天然一份审计底账，
无需沙箱配合。

旁路采集 = 性能、成本、安全三笔账一次算清。

关键指标与告警阈值：把经验变成规则

信号	阈值建议	含义
TTFT 首 token 延迟	**> 2s**	模型推理或上游排队异常
Token 消耗环比	**> 50%**	prompt 膨胀 / 工具死循环
工具调用错误率	**> 5%**	依赖服务或参数生成异常
RAG 相似度 score	**< 0.5**	检索退化，答非所问前兆
HTTP 4xx / 5xx 占比	**> 2%**	外联目标异常或被风控

阈值来自生产实践基线，请按业务调参 —— 指标全部由 OBI 自动产出，无需埋点。

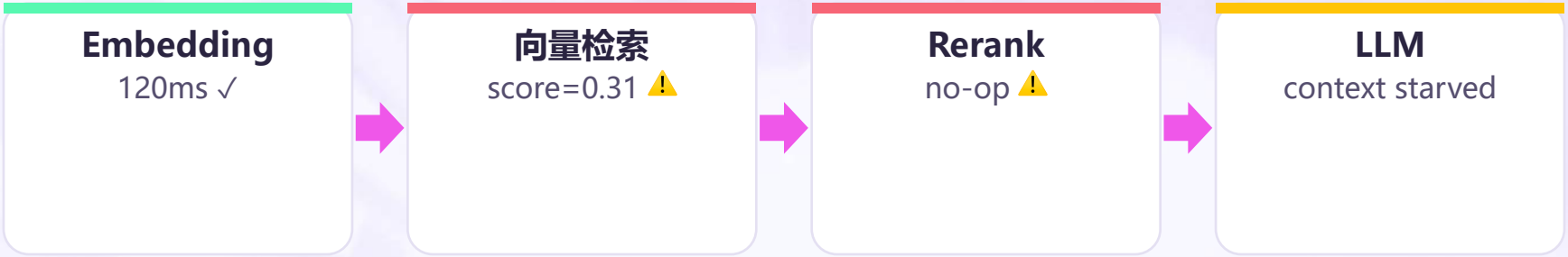
PART FOUR

实战闭环：三个真实问题

04

从「答非所问」到 token 账单，再到沙箱外联审计 ——
—— 用真实案例串起完整链路。

案例一：答非所问，根因不在模型



APM 视角：一次 /chat/completions，2.8s，HTTP 200 —— 其余全是黑盒。

根因：向量 namespace 配错（docs-v3 vs docs-v2），并非模型幻觉。

修复后相似度回升至 0.82+。

动作：namespace 版本校验；hit-count < 3 或 score < 0.5 触发告警。

案例二：token 账单 +320%

现象 AWS Bedrock 输入 token 环比 +320%；传统 APM 不暴露 token 字段，无法归因。

根因 某 KB Agent 把 200 页 PDF 塞进每条 system message（单次调用 80K token）。

动作 切片 + 建索引；设置 system message token 上限；按应用维度配置 token 告警。

OBI 按应用 / 模型拆分 gen_ai.usage.* token 指标，成本第一次可归因。

案例三：沙箱外联审计，旁路即底账

Agentic AICon

沙箱内不可信代码的每次外部访问，被 OBI 旁路记录

L3/L4 流 + 反向 DNS / GeoIP 装饰；无需沙箱配合、无需代码埋点、无需登录现场。

Before

安全团队人肉翻网络日志
逐条比对，漏报靠运气

After

按 Trace 检索沙箱外联
审计底账自动沉淀

隔离负责「敢不敢跑」，可观测负责「跑得明白」—— 两者第一次同时成立。

案例四：裸 HTTP 调 Qwen，Provider 怎么认出来

现象 Python requests 直调 DashScope，没有走任何 SDK —— 传统 APM 只看到一条匿名的 HTTP 200。

姿势一：Host 声明

dashscope.aliyuncs.com 声明为
OpenAI 兼容 Provider
同法支持 vLLM / LiteLLM 网关

姿势二：响应头佐证

提取 X-DashScope-Request-Id
确认 Provider 与请求 ID
与 Host 声明交叉验证

结果 请求归一为 gen_ai 语义 span：model=qwen-max、token 用量、耗时全部可见、可告警。

自建推理与 LLM 网关同理：声明 Host，即纳入 GenAI 视图。

Roadmap：进程 / I/O 提案 #3210

Agentic AICon

① 圈范围
exec / fork / exit
生命周期事件
stdio 字节数 · 文件 I/O

② 定输出
process.* 事件族
进入 OTel 管线
与现有 span 同源导出

③ 管开销
opt-in 选择器
PID 过滤 · 路径白名单
采样控制事件量

④ 分场景部署
runc：节点级
Kata / Firecracker
guest 侧
gVisor：暂不支持

提案 #3210 已提交 open-telemetry 社区 —— **进程与 I/O 将补齐「跑了什么」的最后一块拼图。**

OBI 演进方向：从「进程间通信」走向「进程本身」。

Take away：把 Sandbox 从黑盒变透明

Agentic AICon

1. Sandbox 越隔离越黑盒 —— SDK / 主机级 / 登录排查三条路全断
2. OBI 站在内核与应用之间 —— 零代码、零 SDK、零重启
3. 代码由模型生成的场景 —— 零改造几乎是唯一可行解
4. 运行时边界决定部署 —— runc 节点级，microVM 需 guest 侧
5. GenAI 语义是亮点 —— model / token / tool / MCP 一次采全
6. 一份数据三问全答 —— 性能、成本、安全审计
7. 接入只要三步 —— 装组件、打标签、看数据
8. 可观测是 Agent 基础设施下半场 —— 从「跑得起来」到「跑得明白」

没有可观测的 Sandbox，你不是在运行 Agent，你是在盲盒里赌运气

AgenticAICon

智能体应用与架构工程大会

感谢聆听

关注公众号

更多会议详情